Arduino Workshop

(ESP32 Basic Level)

ดร. วาธิส ลีลาภัทร

ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์

มหาวิทยาลัยขอนแก่น

Hardware : ESP32

ESP32 Key Features

- Xtensa® Dual-Core 32-bit LX6 microprocessors, up to 600 DMIPS
- Integrated 520 KB SRAM
- Integrated 802.11BGN HT40 Wi-Fi transceiver, baseband, stack and LWIP
- Integrated dual mode Bluetooth (classic and BLE)
- 4 MByte flash (32Mbit)

Hardware : ESP32

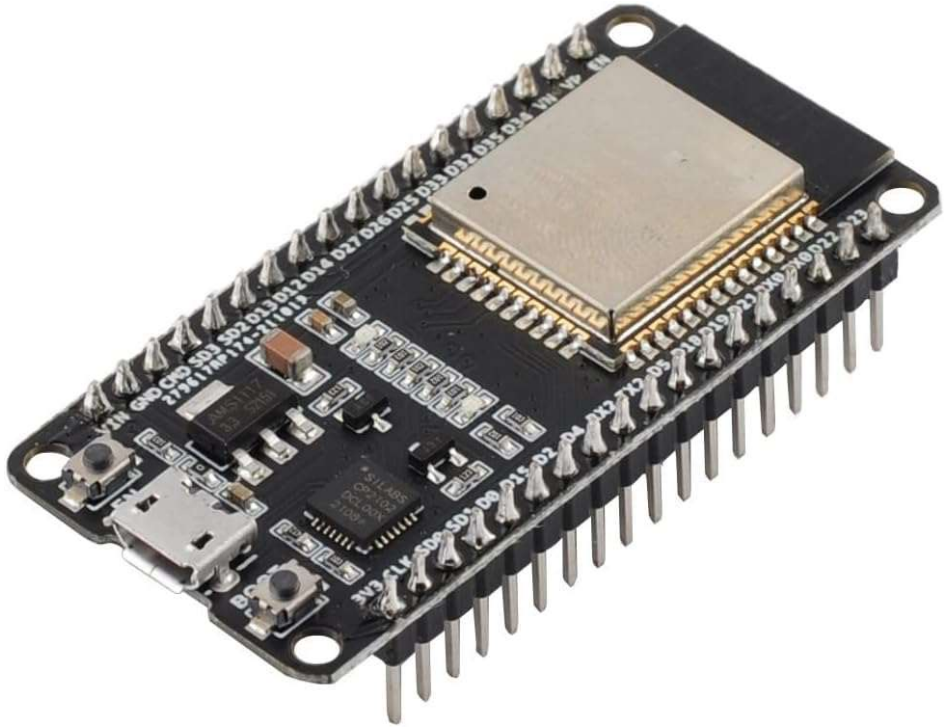
ESP32 Key Features

- Xtensa® Dual-Core 32-bit LX6 microprocessors, up to 600 DMIPS
- Integrated 520 KB SRAM
- Integrated 802.11BGN HT40 Wi-Fi transceiver, baseband, stack and LWIP
- Integrated dual mode Bluetooth (classic and BLE)
- 4 MByte flash (32Mbit)

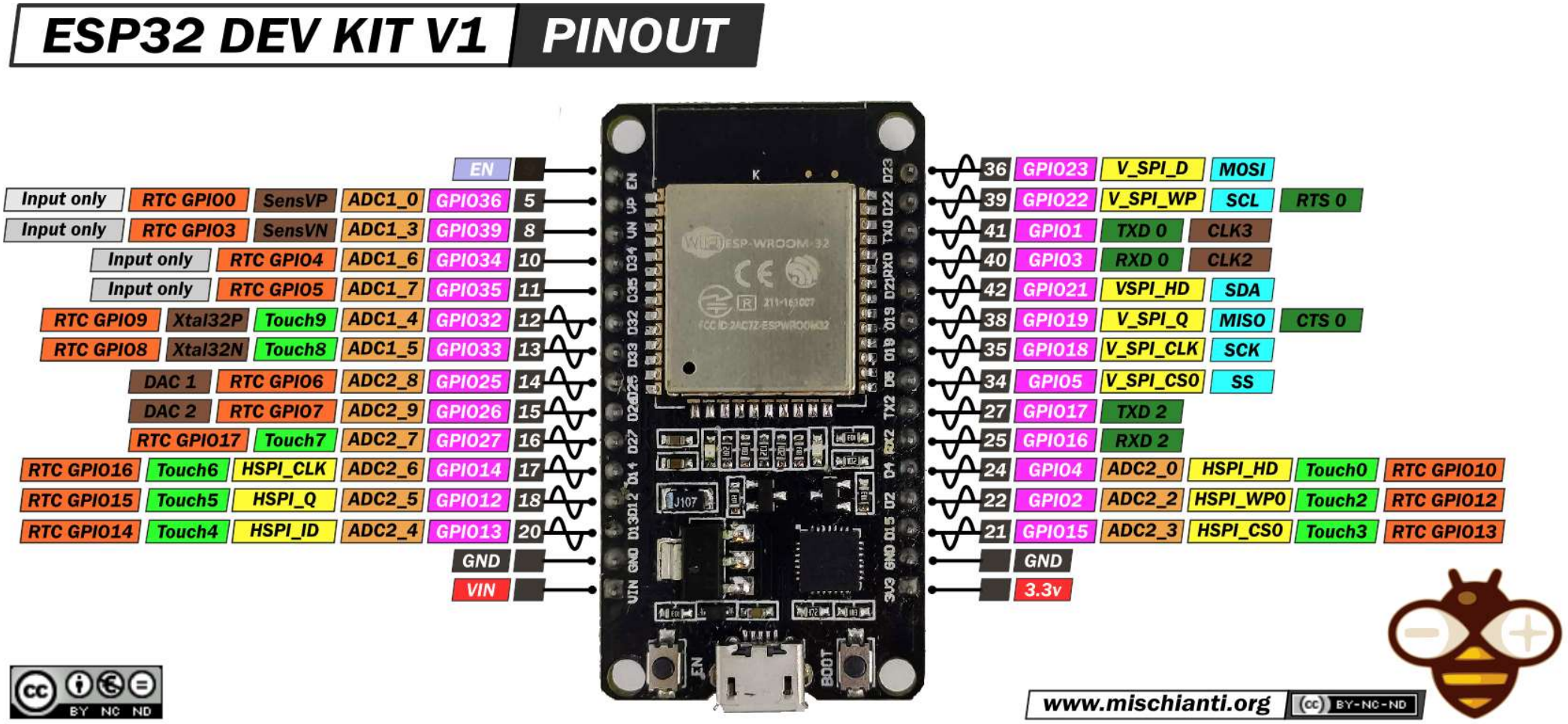
ESP32 DEVKIT

ESP-WROOM-32 DevKit

- utilizes ESP32 chip, 240MHz, RAM 512KB, 4MB flash memory
- WiFi 2.4GHz / Bluetooth



ESP32 DEVKIT



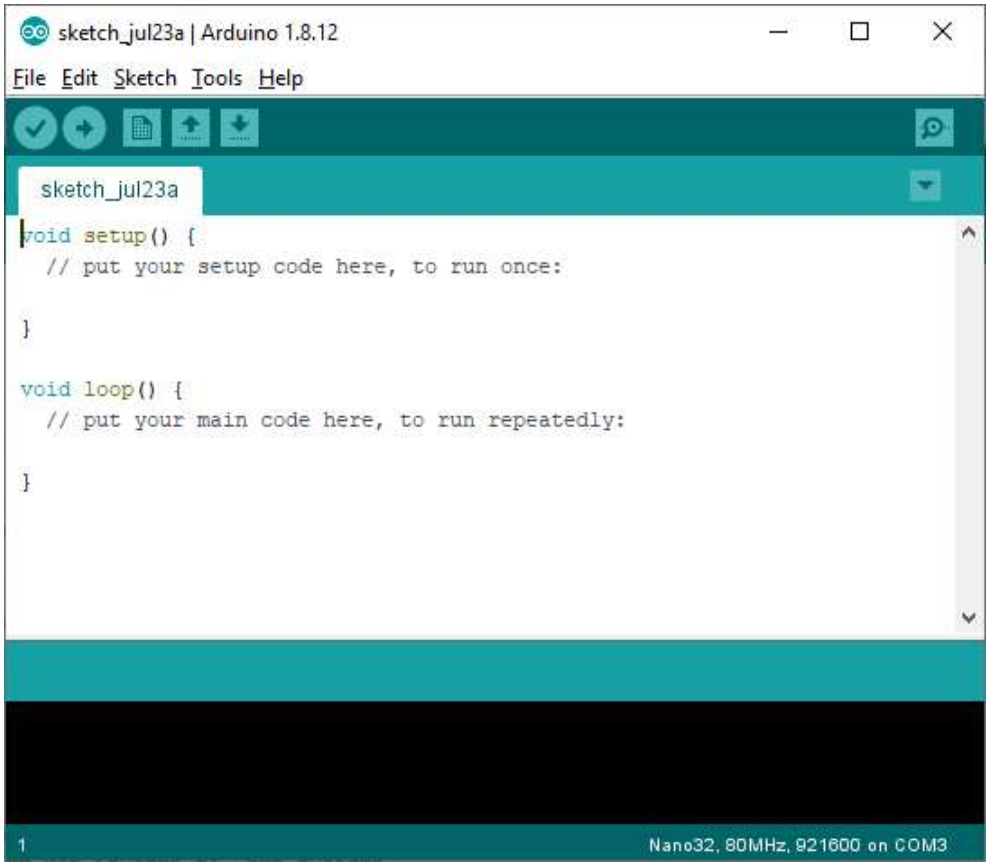
Micro USB Port

Software

Arduino IDE

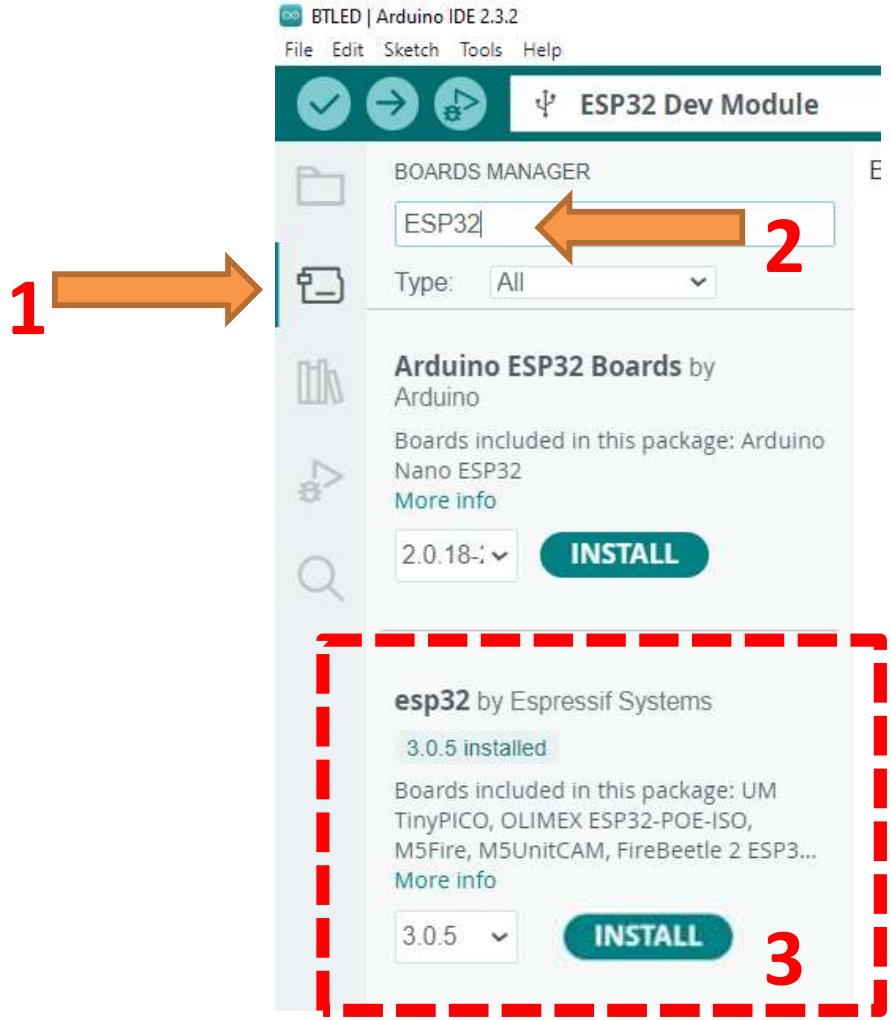
- Use for writing, compiling and downloading code into ESP32 board.

- Go to arduino.cc/en/Main/Software, download & install on your PC.



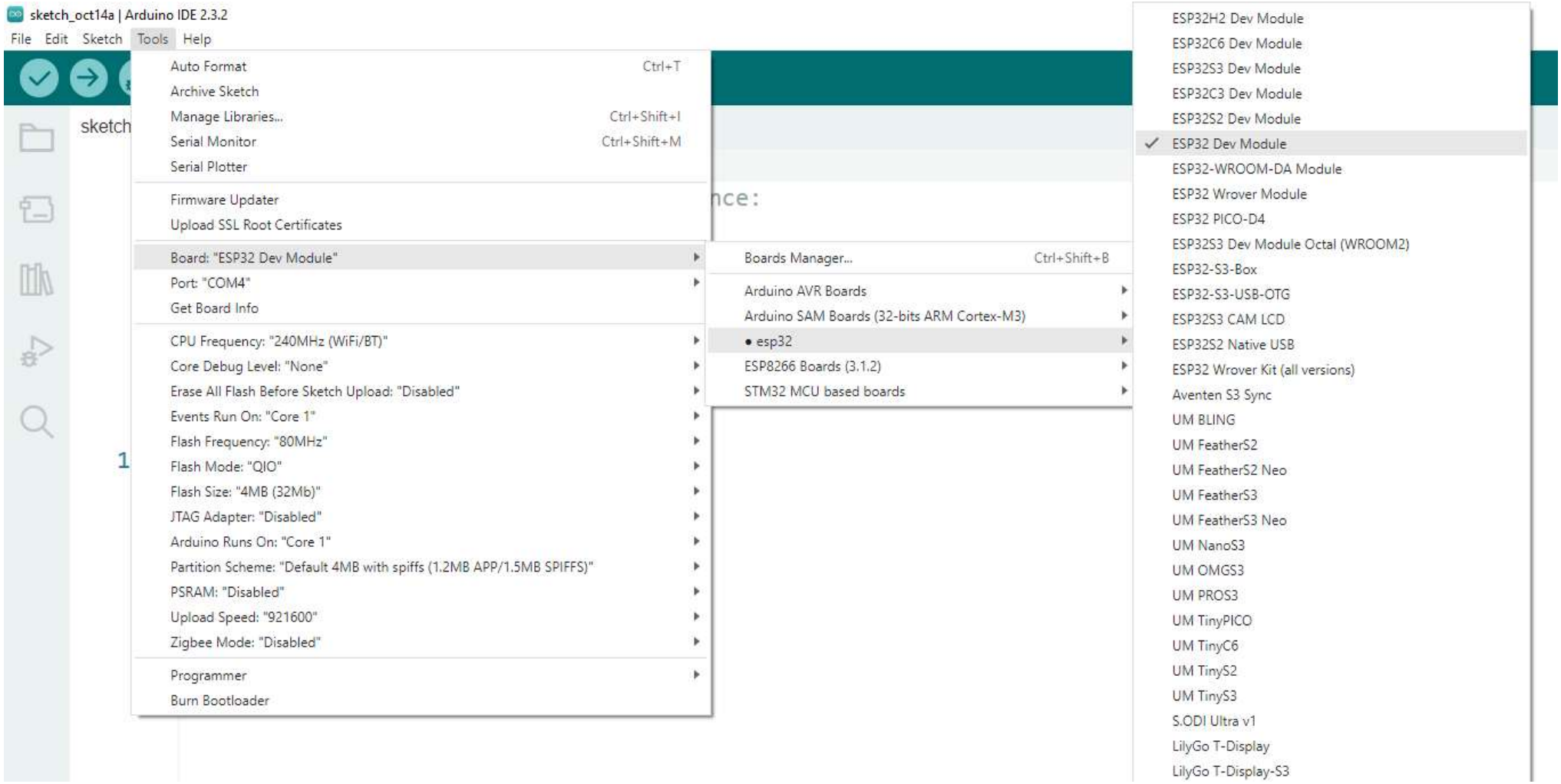
Add ESP32 Board Support

- Click on Board icon on Arduino IDE, type ESP32 in search box
- Look for **esp32 by Espressif Systems**, then click on **INSTALL**



Add ESP32 Board Support

- Go to Tools → **Board: → esp32 → ESP32 Dev Module**
- Go to Tools → **Port: → COM*xx*** (*xx* is a number shown on your PC)



Test ESP32 Board

To test installation:

- Go to: **File → Examples → 01.Basics → Blink**
- At line 24, type:

 # define LED_BUILTIN 2

- Click on download icon 
- You should see the Blue LED blinking on ESP32 Board

```
23  */
24  #define LED_BUILTIN 2
25  // the setup function runs once
26  void setup() {
27    // initialize digital pin LED_
28    pinMode(LED_BUILTIN, OUTPUT);
29  }
```

Arduino Programming

- Software for Arduino boards is developed on Arduino IDE
- Arduino source code is called '**sketch**'
- Sketch utilizes C programming language syntax
- Initial structure of sketch:

```
void setup() {
  // put your setup code here, to run once:

}

void loop() {
  // put your main code here, to run repeatedly:

}
```

Arduino Programming

Example

```
// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH);   // turn the LED on (HIGH is the voltage level)
  delay(1000);                       // wait for a second
  digitalWrite(LED_BUILTIN, LOW);    // turn the LED off by making the voltage LOW
  delay(1000);                       // wait for a second
}
```

Digital I/O Port

pinMode()

Description

Configures the specified pin either as an input or an output.

Syntax

pinMode(*pin*, *mode*)

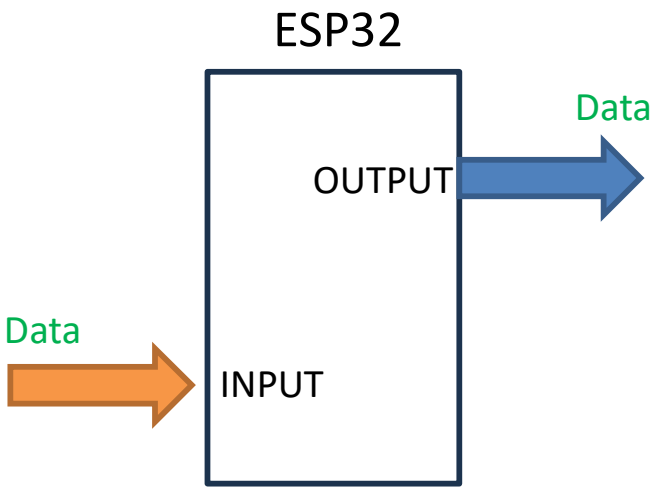
Parameters

pin: the number of the pin whose mode you wish to set

mode: INPUT, OUTPUT

Returns

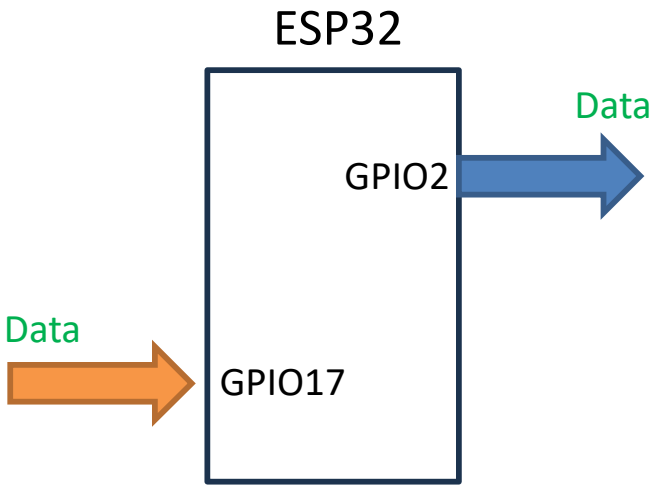
None



Digital I/O Port

Example: pinMode()

```
setup () {
    pinMode(2, OUTPUT);       // GPIO2 is output
    SWPin = 17;               // assign value 17 to variable SWPin
    pinMode(SWPin, INPUT);    // now GPIO 17 is input
}
```



Digital I/O Port

digitalWrite()

Description

Write a HIGH or a LOW value to a digital pin. If the pin has been configured as an OUTPUT with pinMode()

Syntax

digitalWrite(*pin*, *value*)

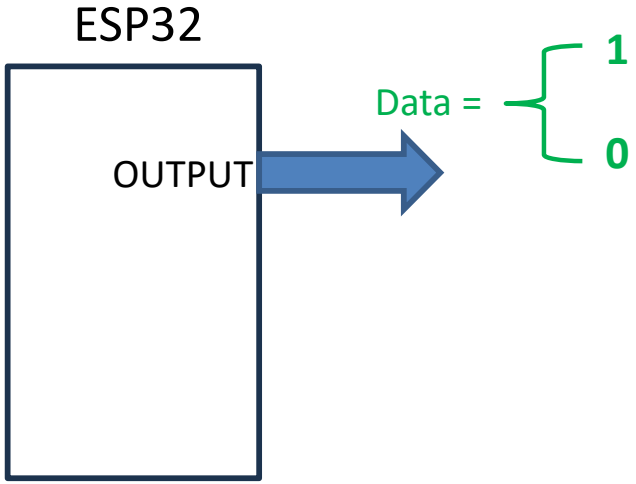
Parameters

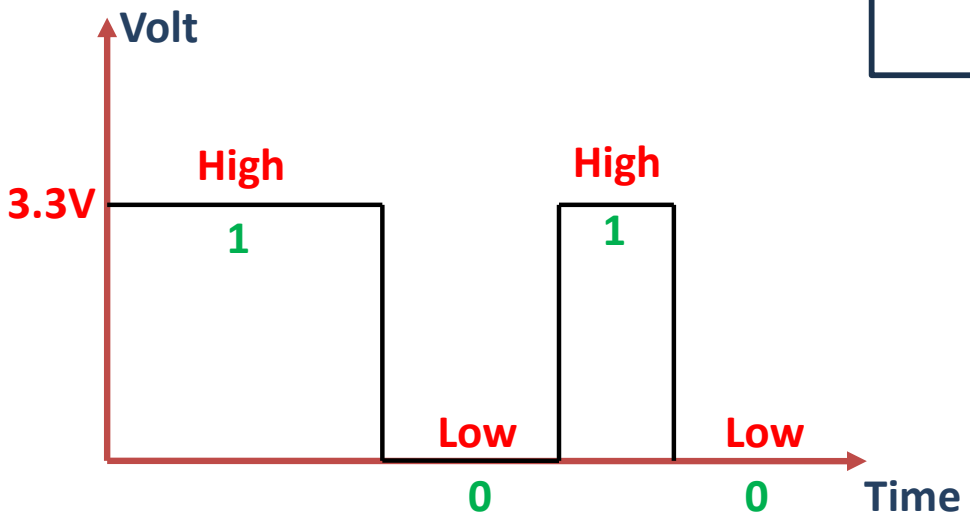
pin: the pin number

value: HIGH or LOW

Returns

none

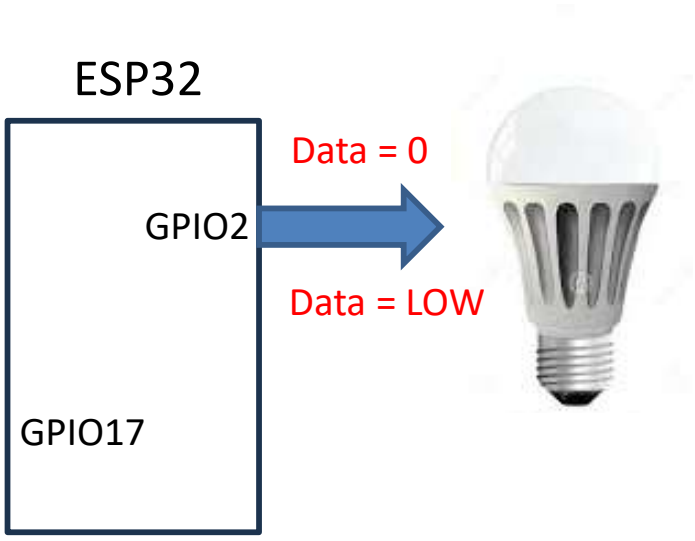


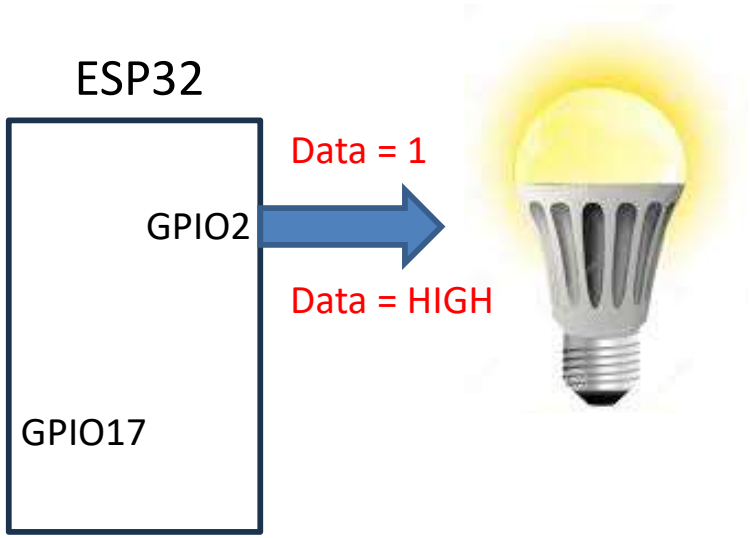


Digital I/O Port

Example: digitalWrite()

```
setup () {
    pinMode(2, OUTPUT);          // GPIO2 is output
    digitalWrite(2, HIGH);       // Supply 3.3 Volt to GPIO2
    delay(1000);                 // Suspend for 1000 milisecond
    digitalWrite(2, LOW);        // Supply 0 Volt to GPIO2
}
```





Digital I/O Port

digitalRead()

Description

Reads the value from a specified digital pin, either HIGH or LOW.

Syntax

digitalRead(*pin*)

Parameters

pin: the pin number

Returns

HIGH or LOW.

Digital I/O Port

Example

Read status of input pin 7 and display on pin 13.

```
int ledPin = 13;   // LED connected to digital pin 13
int inPin = 7;     // pushbutton connected to digital pin 7
int val = 0;       // variable to store the read value

void setup()
{
  pinMode(ledPin, OUTPUT);      // sets the digital pin 13 as output
  pinMode(inPin, INPUT);        // sets the digital pin 7 as input
}

void loop()
{
  val = digitalRead(inPin);     // read the input pin
  digitalWrite(ledPin, val);    // sets the LED to the button's value
}
```

Programming Structures

for

Description

The for statement is used to repeat a block of statements enclosed in curly braces. An increment counter is usually used to increment and terminate the loop. The for statement is useful for any repetitive operation

Syntax

```
for (initialization; condition; increment) {
    // statement(s);
}
```

Example

```
int count = 0;  // initialize variable

void setup() {
    Serial.begin(9600);
}
void loop() {
    for (int i = 0; i <= 255; i++) {
        count = Count+2;
        Serial.println(Count);
    }
}
```

Programming Structures

if

Description

The if statement checks for a condition and executes the proceeding statement or set of statements if the condition is 'true'.

Syntax

```
if (condition) {
    // statement(s);
}
```

```
if (condition1) {
    // do statement group A
}
else if (condition2) {
    // do statement group B
}
else {
    // do statement group C
}
```

Example

```
int a, b, c;
int b, c;
if (a > 0) {
    c = a + b;
} else if (a == 0) {
    c = b;
} else {        //a must be less than zero
    c = a);
}
```

Programming Structures

while

Description

A while loop will loop continuously, and infinitely, until the expression inside the parenthesis, () becomes false.

Syntax

```
while (condition) {
    // statement(s);
}
```

Example

```
var = 0;
while (var < 200) {
    // do something repetitive 200 times
    var++;
    Serial.println(var);
}
```

Data types

boolean	‘true’, ‘false’
char	8-bit value from -128 to 127
unsigned char	8-bit value from 0 to 255
byte	same as *unsigned char*
int	16-bit value from -32,768 to 32,767
unsigned int	16-bit value from 0 to 65535
word	same as *unsigned int*
long	32-bit value from -2,147,483,648 to 2,147,483,647
unsigned long	32-bit value from 0 to 4,294,967,295
short	same as *int*
float	32-bit value -3.4028235E+38 to 3.4028235E+38
double	same as *float*
string	array of character

Built-in functions

Digital I/O
- pinMode()
- digitalWrite()
- digitalRead()

Analog I/O
- analogReference()
- analogRead()
- analogWrite() - PWM

Advanced I/O
- tone()
- noTone()
- shiftOut()
- shiftIn()
- pulseIn()

Time
- millis()
- micros()
- delay()
- delayMicroseconds()

Math
- min()
- max()
- abs()
- constrain()
- map()
- pow()
- sqrt()

Communication
- Serial
- Stream

Serial Print

Serial.print() & println()

Description

Prints data to the serial port as human-readable ASCII text followed by a carriage return character (ASCII 13, or '\r') and a newline character (ASCII 10, or '\n'). These commands are useful for communication with PC and debugging.

Syntax

Serial.println(val)
Serial.println(val, format)

Example

```
int analogValue = 0;    // variable to hold the analog value
void setup() {
      Serial.begin(9600);      // open the serial port at 9600 bps:
}
void loop() {
      analogValue = analogRead(0); // read the analog input on pin 0:
      // print it out in many formats:
      Serial.println(analogValue);       // print as an ASCII-encoded decimal
      Serial.println(analogValue, DEC);  // print as an ASCII-encoded decimal
      Serial.println(analogValue, HEX);  // print as an ASCII-encoded hexadecimal
      Serial.println(analogValue, BIN);  // print as an ASCII-encoded binary
}
```

Serial Print

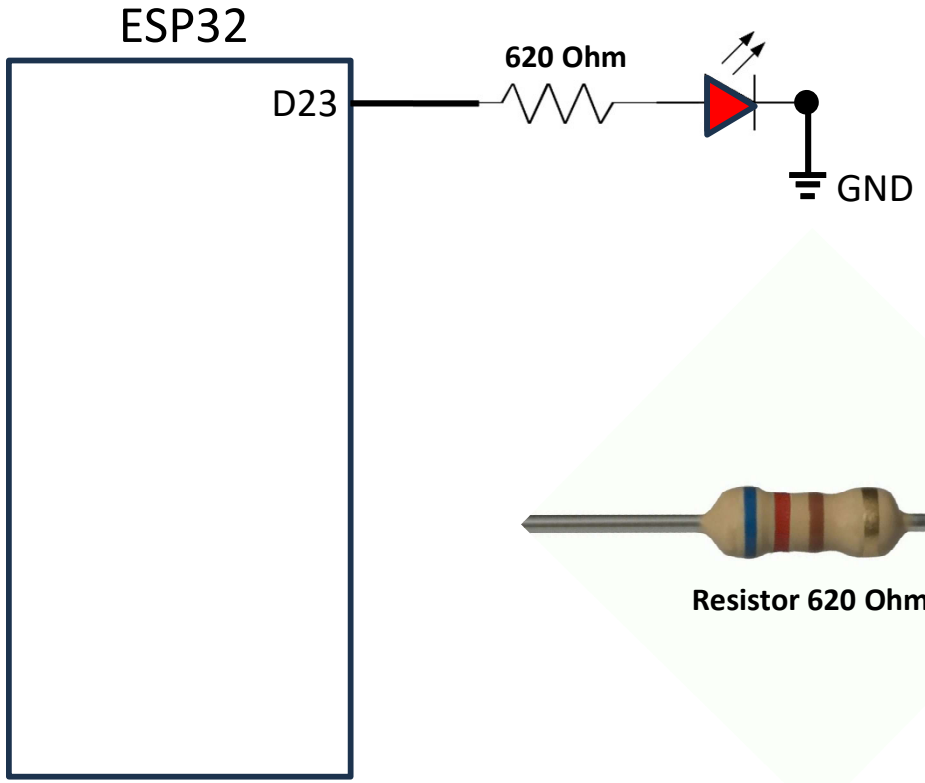
- Open serial port monitor (Menu **Tools→Serial Monitor**)
- Run the following sketch

```
void setup()
{
        Serial.begin(9600);
        Serial.println("Hello World");
}
void loop()
{
}
```

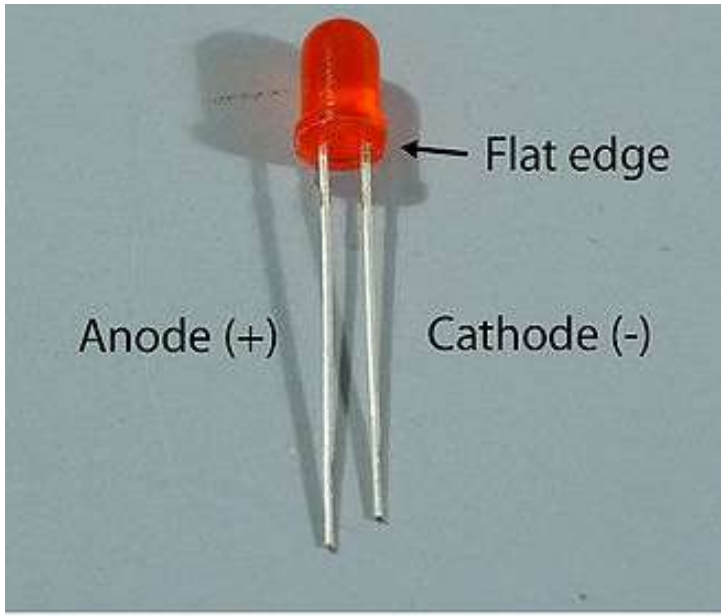
ESP32 Input/Output

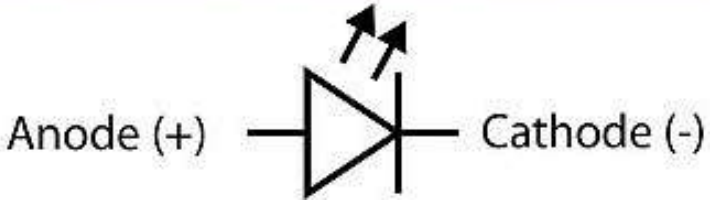
Example 1: Blinking external LED

- Open blink example: *File → Examples → 01. Basics → Blink*
- Replace **LED_BUILTIN** with number **23**
- Connect resistor and LED to ESP32 at Pin D23



Resistor 620 Ohm





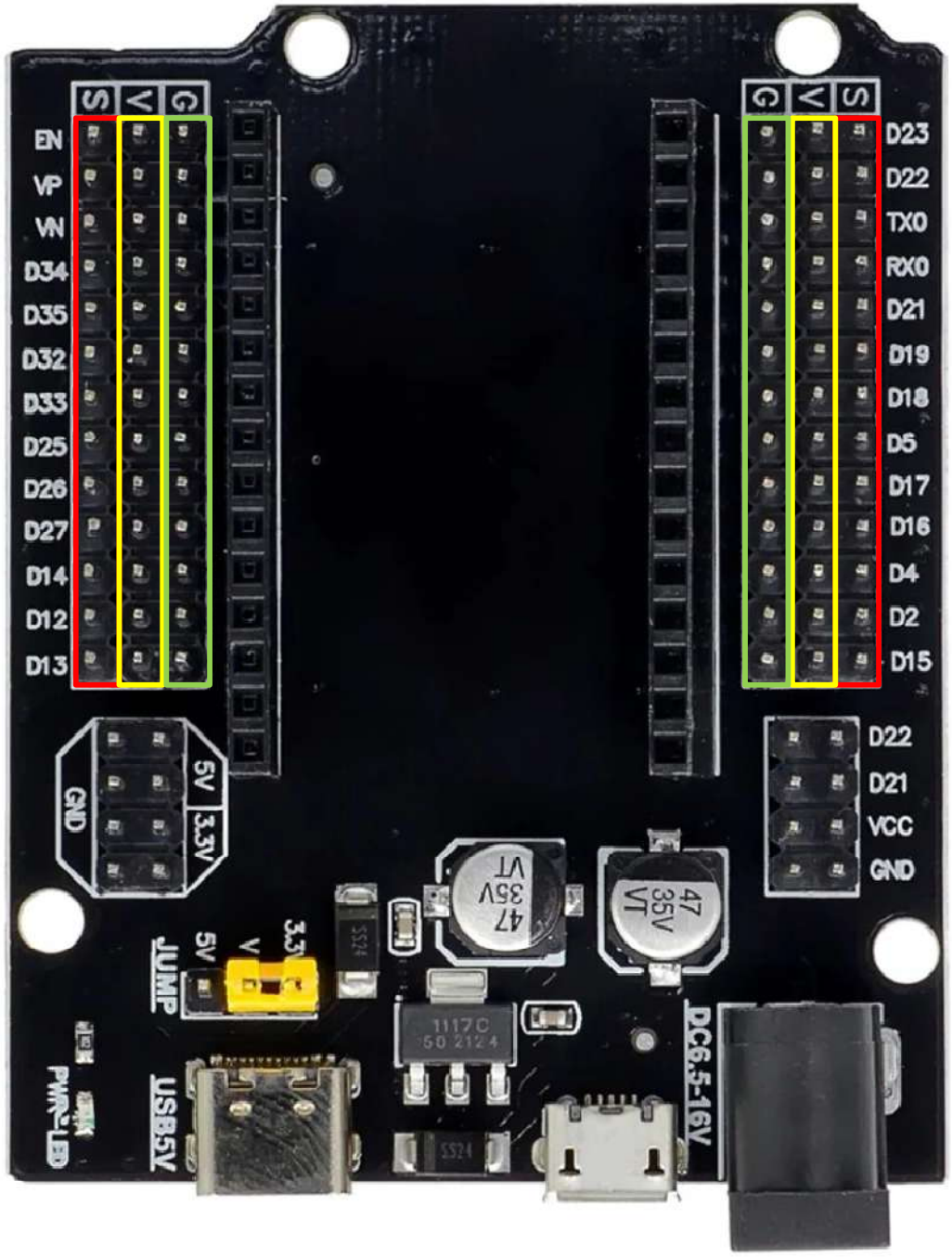
ESP32 Extension Board

- Pins extension
- External power supply

S (Signal) Pins

V (3V Power supply) Pins

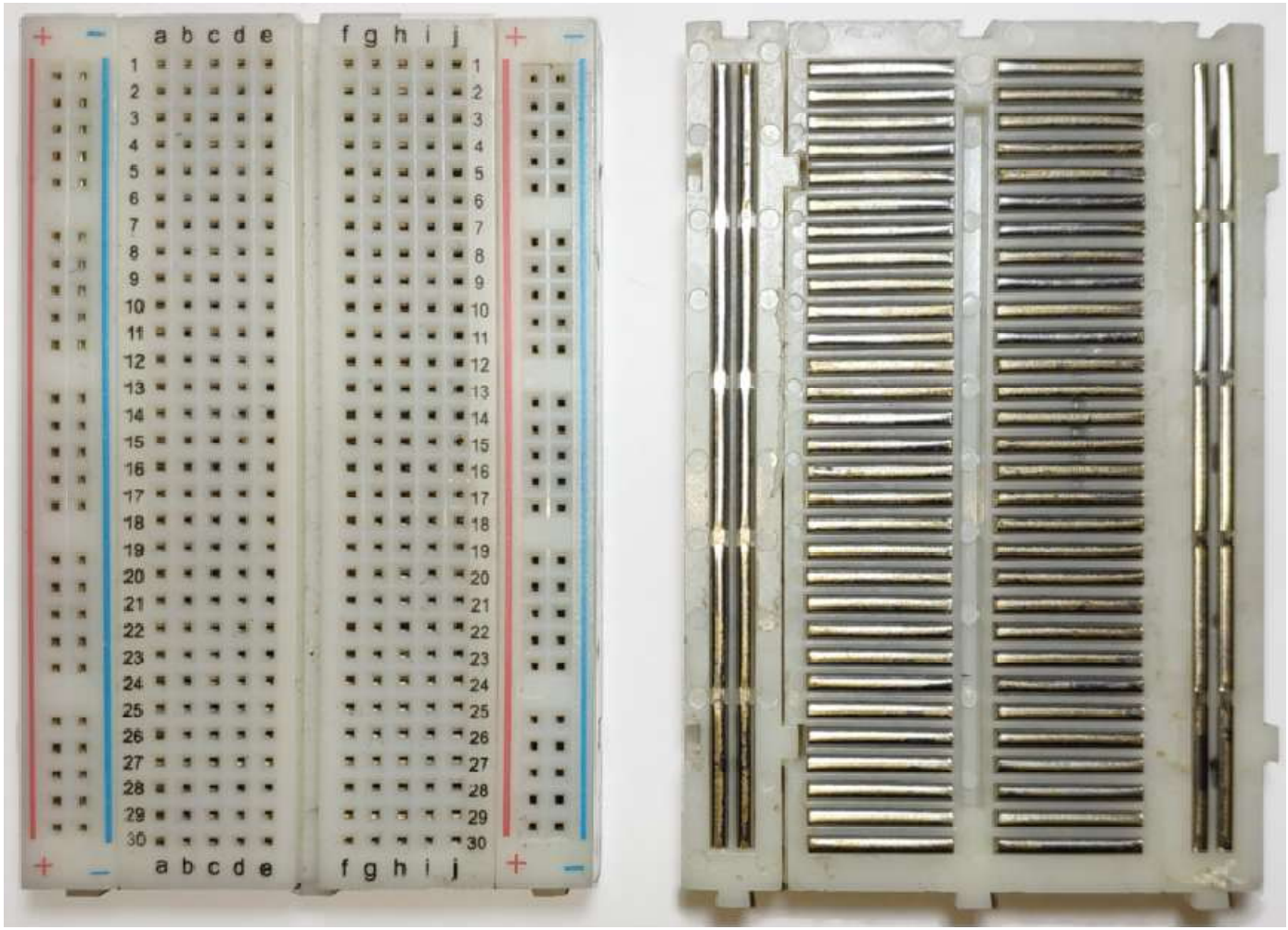
G (Ground) Pins



Circuit Construction

Prototyping board

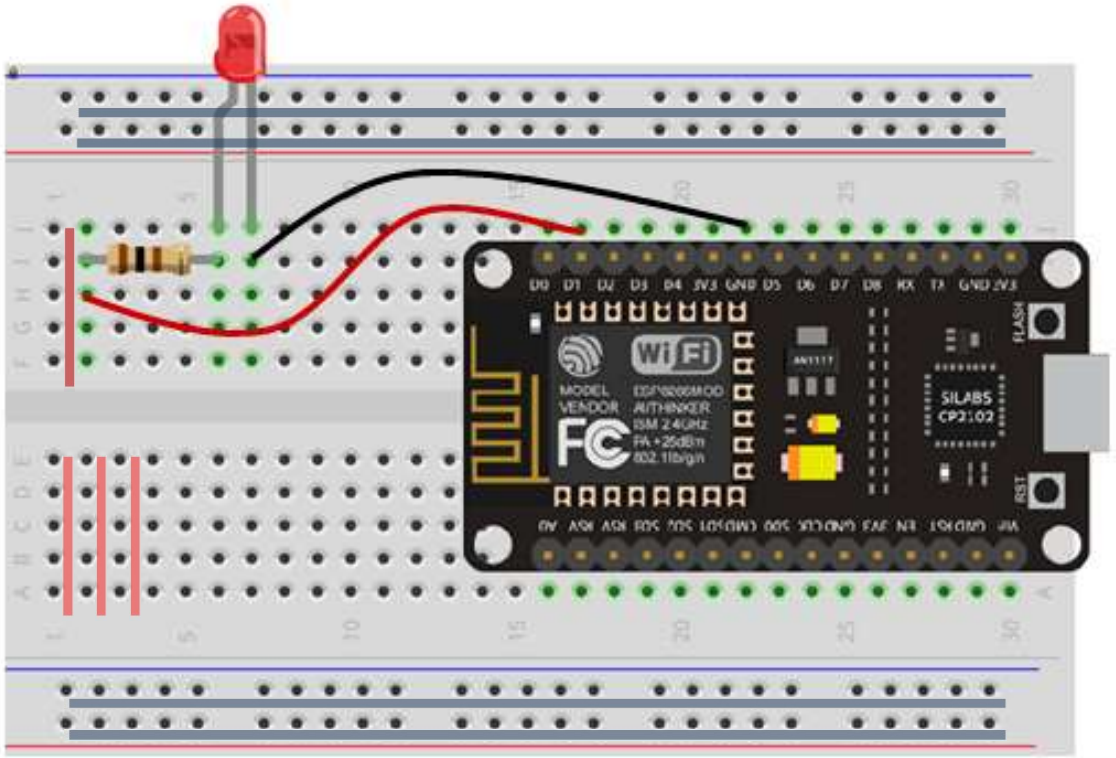
- Pre-connected conductor stripes
- Horizontal stripes on 2 top rows and 2 bottom rows
- Vertical stripes in the middle



Circuit Construction

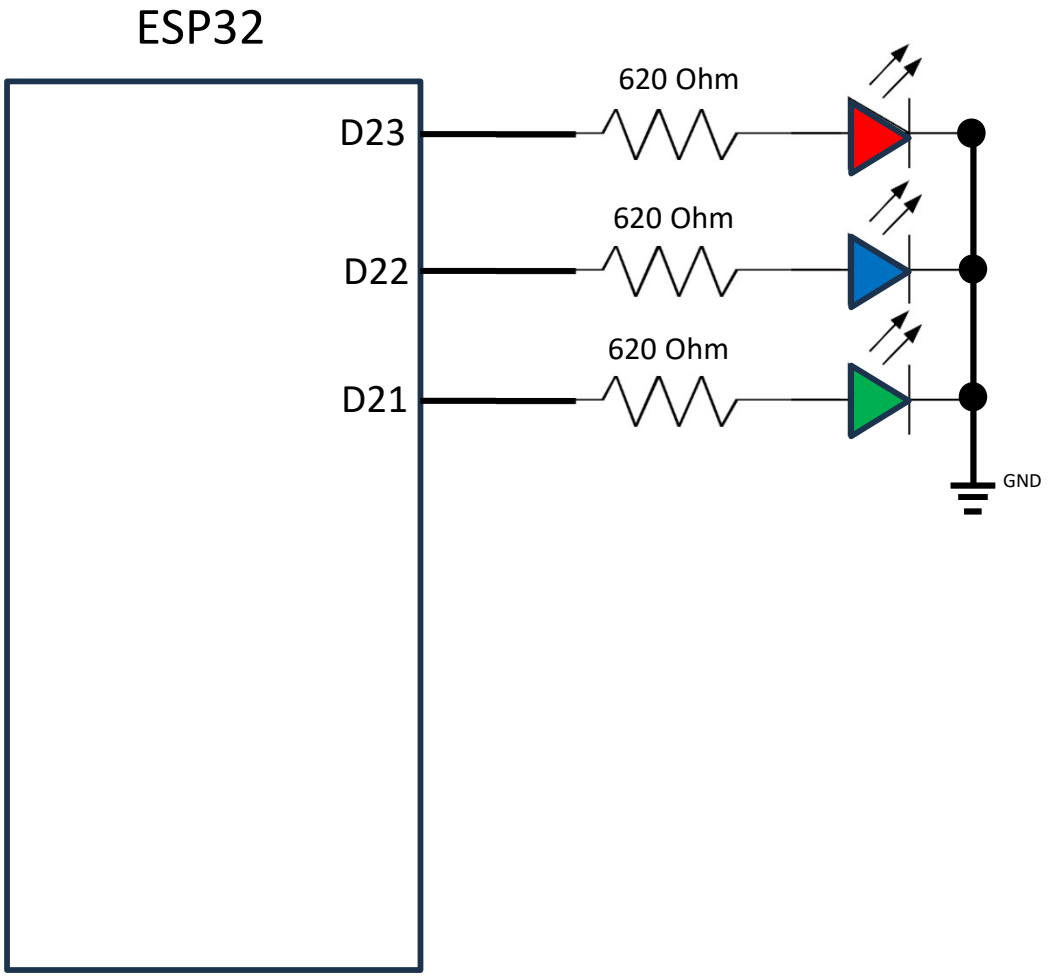
Prototyping board

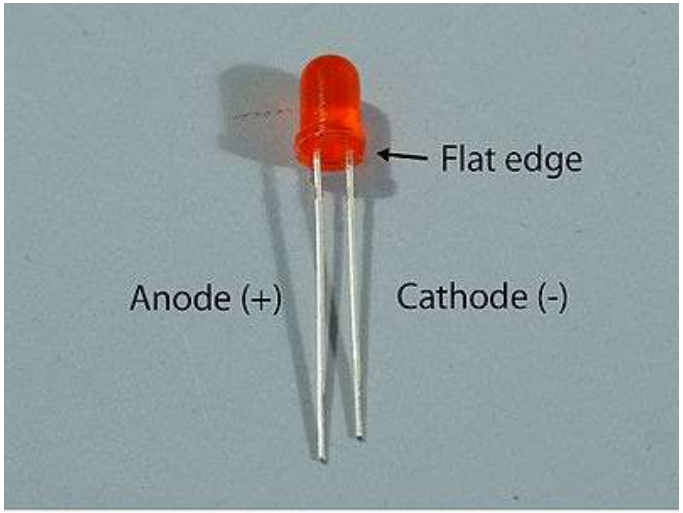
- Pre-connected conductor stripes
- Horizontal stripes on 2 top rows and 2 bottom rows
- Vertical stripes in the middle

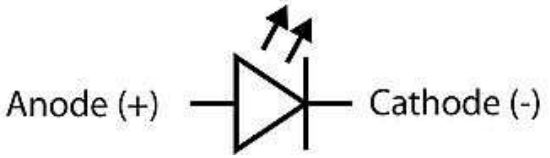


ESP32 Input/Output

Exercise 1: Blinking 3 external LEDs



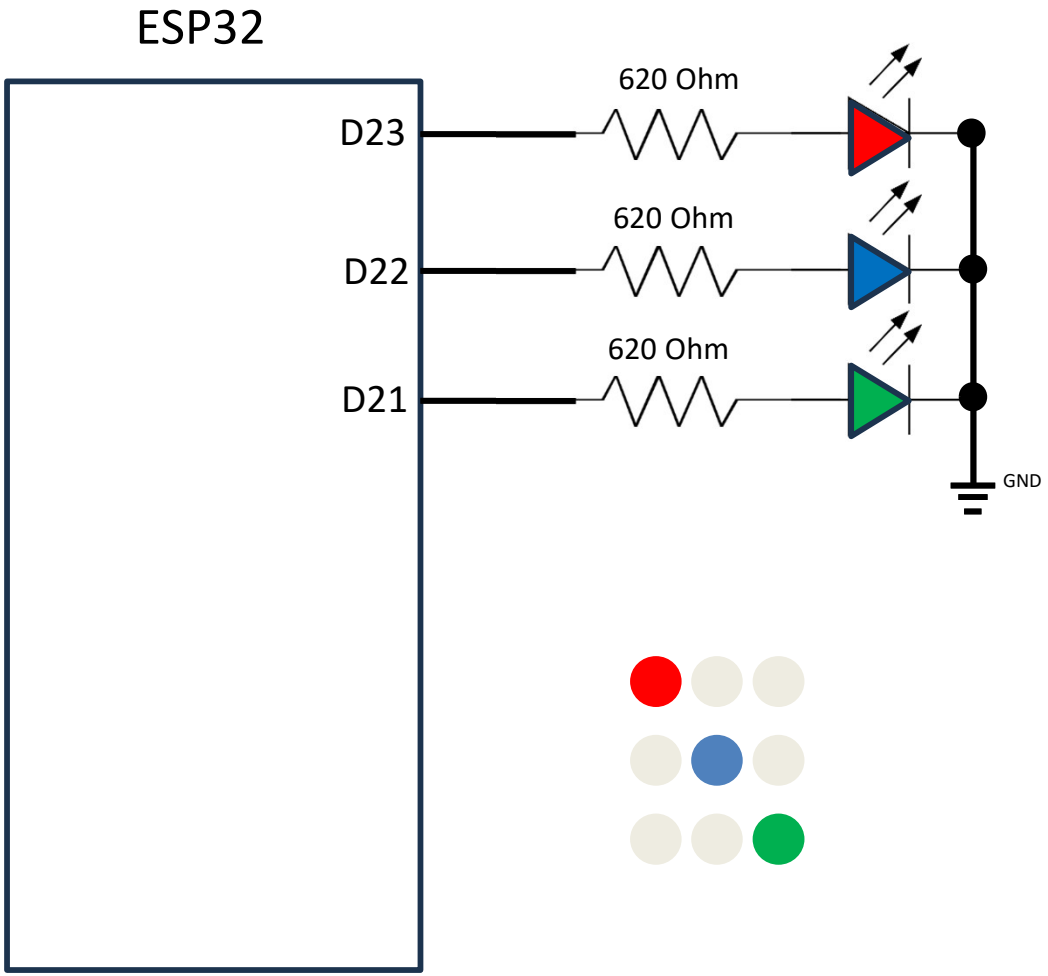


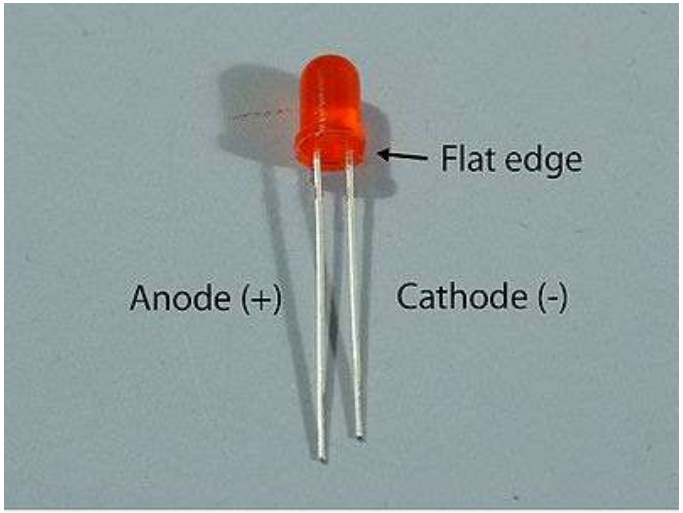


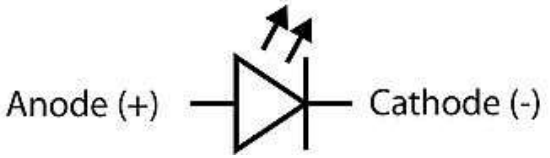
ESP32 Input/Output

Exercise 2: Chasing LEDs

Modify your sketch to make chasing LEDs



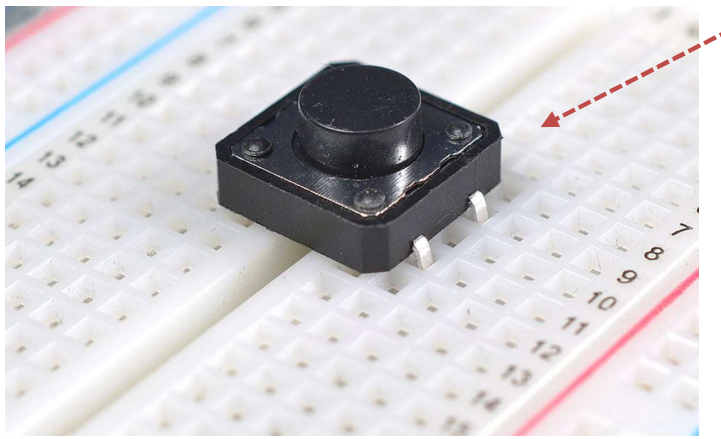


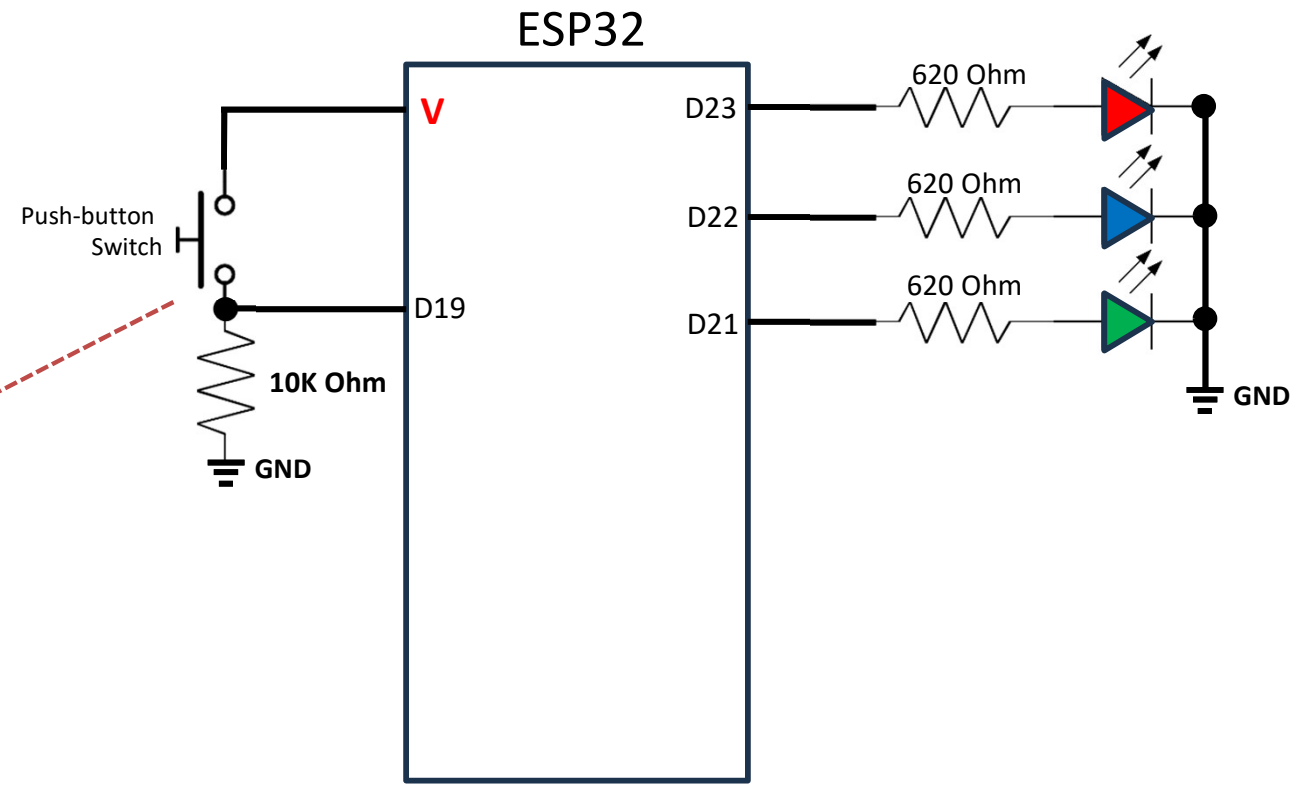


ESP32 Input/Output

Example 2: Reading digital input

- Open example: *File → Examples → 01. Basics → DigitalReadSerial*
- Change ***int pushButton = 2*** to ***int pushButton = 19***
- Change delay value in ***delay(1)*** to ***delay(10)***

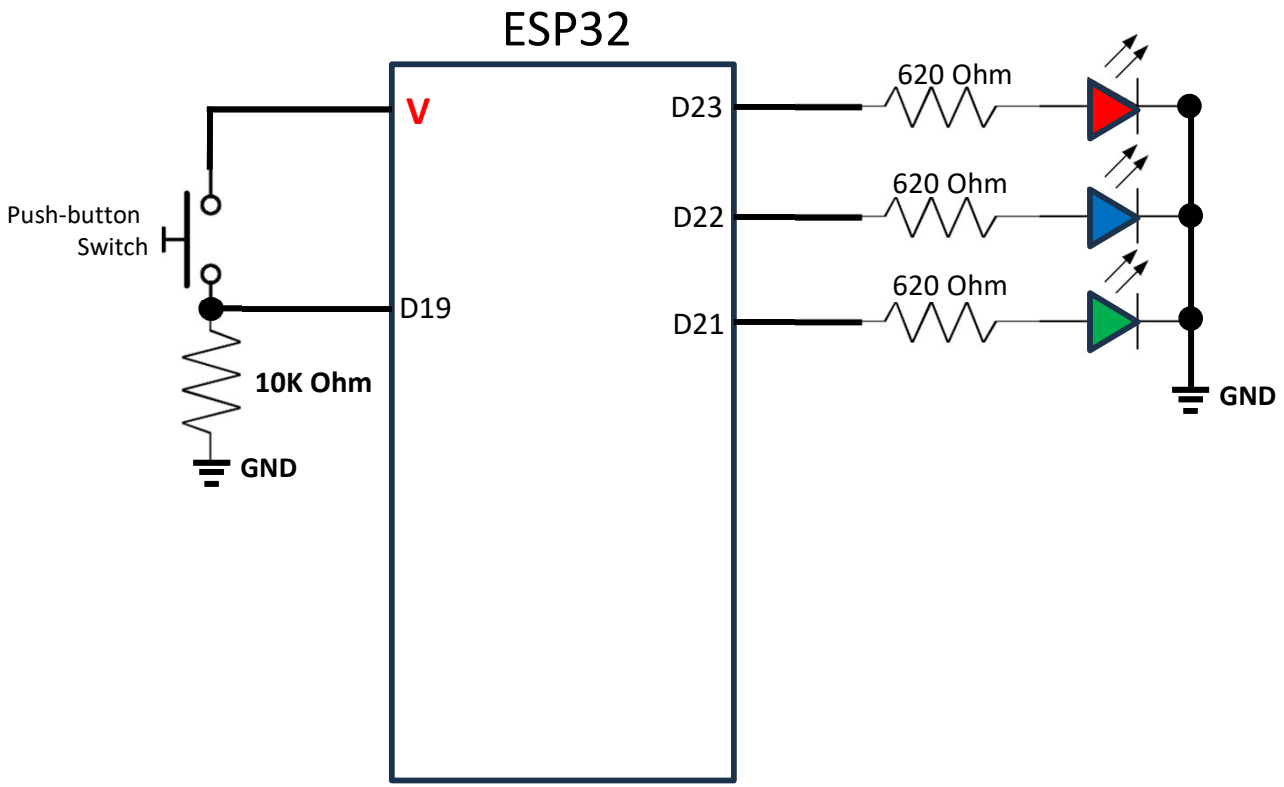




ESP32 Input/Output

Exercise 3 : Combine input & output

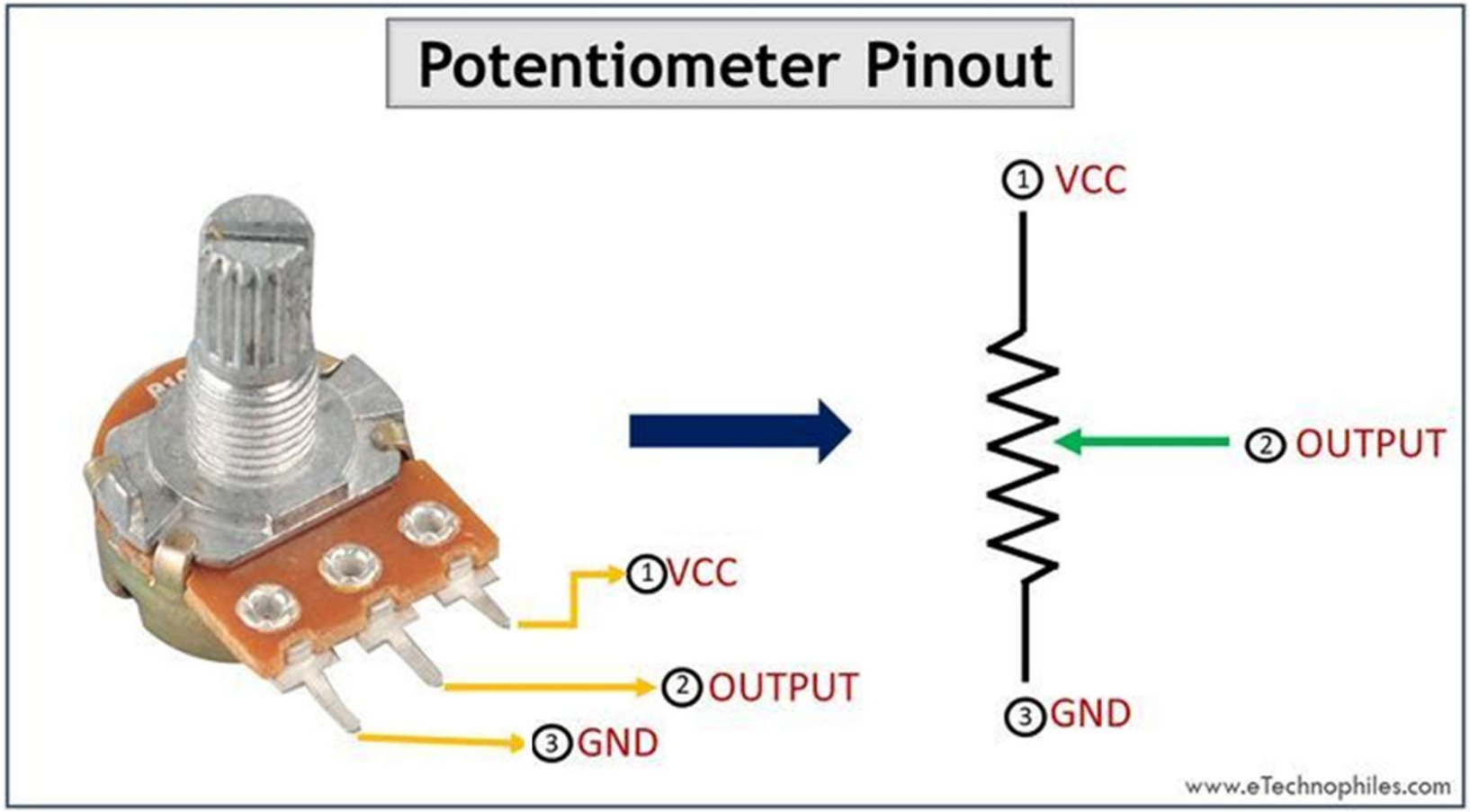
- LED is **on** when button is pressed and **off** when button is released
- LED if **off** when button is pressed and **on** when button is released
- LED blinks **5 times** when button is pressed
- ***Reverse*** chasing direction when button is pressed



ESP32 Analog Input

Simple analog input

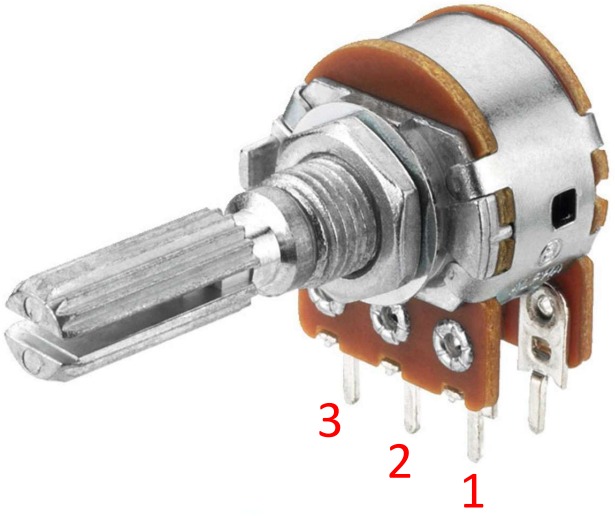
- Potentiometer is used as analog voltage source
- Analog voltage is converted to number ranging from 0 – 1023 (0 – 3.3v)

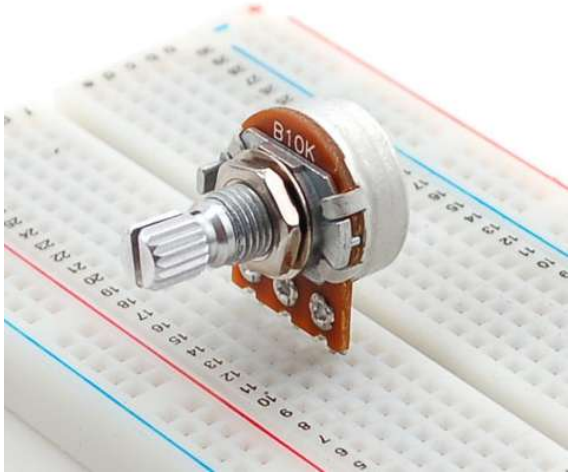


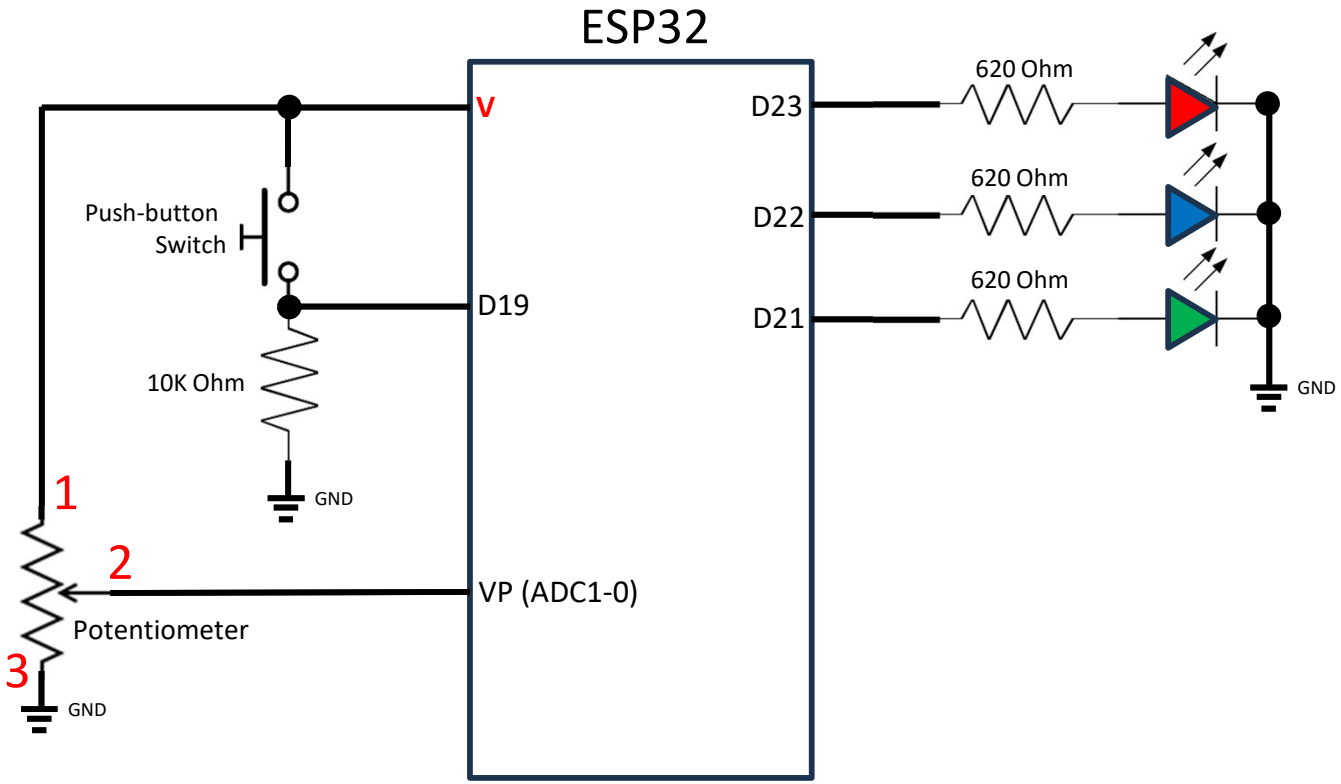
ESP32 Analog Input

Example 3 : Simple analog input

- Potentiometer is used as analog voltage source
- Analog voltage is converted to number ranging from 0 – 1023 (0 – 3.3v)







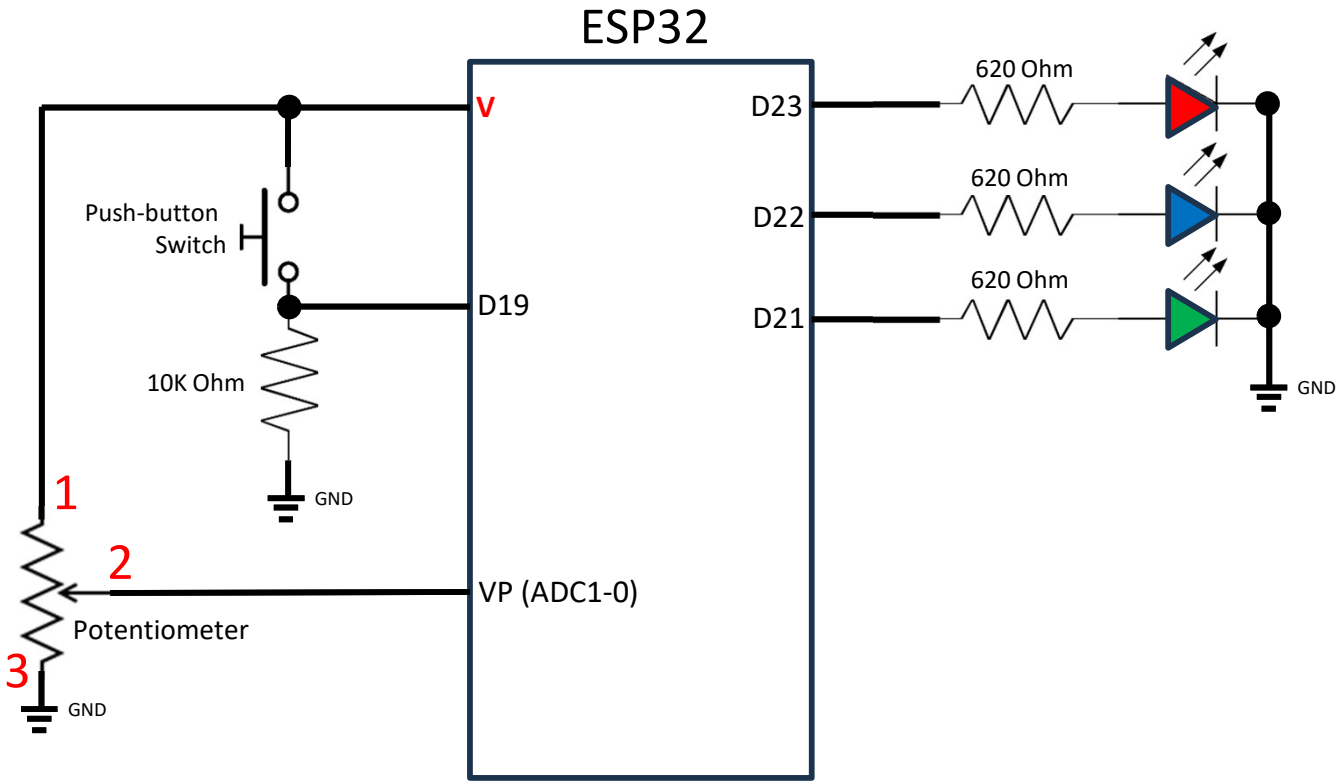
ESP32 Analog Input

```
int POT = 36;    // select the input pin for the potentiometer
int analogValue = 0;  // variable to store the value coming from the sensor

void setup() {
   Serial.begin(9600);
}

void loop() {
  analogValue = analogRead(POT); // read the value from the sensor:
  Serial.println(analogValue);
  delay(500);
}
```

ESP32 Analog Input

Example 4

- Open default blink example: *File → Examples → 03. Analog → AnalogInput*
- Change ***int sensorPin = A0*** to ***int sensorPin = 36***
- Make D23, D22, D1 to output
- Adjust potentiometer to change blinking frequency



ESP32 Analog Input

Exercise 4: Combine analog & digital inputs

Modify your chasing LEDs sketch to make it:

- chasing speed controllable using potentiometer
- chasing direction switchable using push-button

DHT11 Digital Temperature & Humidity Sensor

Specifications:

- Temperature 0-50°C
- Humidity 20-80%

DHT11 Digital Temperature & Humidity Sensor

Blinking 3 external LEDs